Foundations of Statistical Learning & Algorithms

Today's goal • 3/3 items

Module 1
Introduction to Linear Algebra and Vector Spaces

Module 2
Linear Transformation and Matrix Decomposition

Module 3
Probability Foundations for Statistical Learning

Module 4
Introduction to Model Evaluation and Optimization

Lesson 1: Model Selection Techniques
- Introduction to Model Selection — Reading • 5 min
- Bayesian Model Selection — Reading • 30 min
- Bayesian Model Selection Readings — Reading • 1h

Lesson 2: Cross Validation
- Introduction to Cross-Validation — Reading • 25 min
- K-Fold Cross-Validation — Reading • 2 min
- Leave-One-Out Cross-Validation (LOOCV) — Reading • 10 min

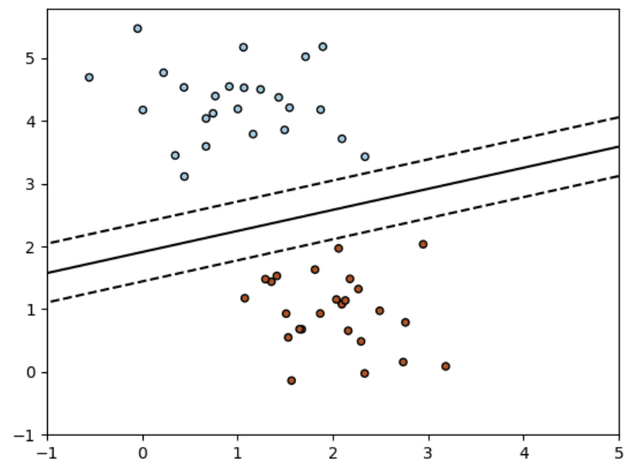
Lesson 3: Introduction to Optimization Techniques
- Introduction to Optimization Techniques — Reading • 13 min
- Optimization Using Gradient Descent — Reading • 40 min
- Gradient Descent with Momentum — Reading • 45 min
- **Stochastic Gradient Descent (SGD)** — Reading • 40 min
- Constrained Optimization — Reading • 16 min
- Lagrange Multipliers — Reading • 8 min
- Convex Optimization — Reading • 8 min
- Linear Programming — Reading • 15 min
- Quadratic Programming — Reading • 10 min
- Check Your Knowledge: Optimization Techniques — Graded Assignment • 15 min

Stochastic Gradient Descent (SGD)

0:08 / 2:56 2x

Stochastic Gradient Descent (SGD) is a powerful optimization method used extensively in machine learning, particularly for large-scale problems. The key idea behind SGD is to approximate the gradient of the objective function using a small subset of data rather than computing it over the entire dataset. This approximation significantly reduces the computational cost, making it feasible to optimize models with large datasets and complex functions.

Resource: SciKit: Stochastic Gradient Descent (25 minutes)



Why Use Approximate Gradients?

Computing the exact gradient of an objective function can be very time-consuming, especially when the dataset is large. In many machine learning problems, the objective function $L(\theta)$ is often expressed as a sum of individual loss functions $L_n(\theta)$, each corresponding to a single data point:

$$L(\theta) = \sum_{n=1}^{N} L_n(\theta)$$

Here, θ represents the model parameters that we aim to optimize. The exact gradient of $L(\theta)$ is computed by summing the gradients of all individual loss functions:

$$\theta_{i+1} = \theta_i - \gamma_i \nabla_\theta L\left(\theta_i\right) = \theta_i - \gamma_i \sum_{n=1}^{N} \nabla_\theta L_n\left(\theta_i\right)$$

However, when the training set is very large, computing this sum at each iteration can be computationally expensive. This is where SGD offers a practical solution by approximating the gradient.

Stochastic Gradient Descent (SGD)

In SGD, instead of computing the gradient over the entire dataset, we approximate it using a single randomly selected data point (or a small subset of data points, known as a mini-batch). The update rule for the parameters in SGD is:

$$\theta_{i+1} = \theta_i - \gamma_i \nabla_\theta L_n\left(\theta_i\right)$$

Where $\nabla_\theta L_n(\theta_i)$ is the gradient of the loss function for a single data point n. The gradient computed from a single data point is a noisy estimate of the true gradient, but it still generally points in the direction of the optimal solution.

Mini-Batch Gradient Descent

A compromise between batch gradient descent and SGD is mini-batch gradient descent, where the gradient is computed using a small, randomly selected subset of the data, typically ranging from 16 to 256 data points. This approach provides a balance between the computational efficiency of SGD and the stability of batch gradient descent.

- **Large Mini-Batches:** Using a large mini-batch size provides a more accurate estimate of the gradient, which reduces the variance in the parameter updates and leads to more stable convergence. However, each update is more computationally expensive.
- **Small Mini-Batches:** Small mini-batches are quicker to compute and introduce more noise into the gradient estimates. While this noise can help the algorithm escape local minima, it can also lead to more erratic convergence behavior.

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →